

Illusions of Performance: Train–Test Leakage in Similarity Search Benchmarks

Diya Gupta

*Chitkara University Institute of
Engineering & Technology,
Chitkara University,
Punjab, India*
diya303.be22@chitkara.edu.in

Aakanksha Sharma

*Chitkara University Institute of
Engineering & Technology,
Chitkara University,
Punjab, India*
aakanksha6.be22@chitkara.edu.in

Siya Bansal

*Chitkara University Institute of
Engineering & Technology,
Chitkara University,
Punjab, India*
siya2386.be22@chitkara.edu.in

Swati Goel

*Chitkara University Institute of
Engineering & Technology,
Chitkara University,
Punjab, India*
swati.goel@chitkara.edu.in

Abstract—Similarity search systems based on dense vector embedding and approximate nearest neighbor (ANN) indexing have become an integral part of modern information retrieval, recommendation, and question answering systems. Currently, it is easy to evaluate such systems by using publicly available benchmarks such as Recall@K and Mean Reciprocal Rank (MRR). But a significant problem in benchmarking is train-test leakage in which the test query or its similar-sounding counterpart is accidentally used in the search results. When this occurs, the retrieval performance degrades in ways that are difficult to attribute to system failure as the leakage vectors compete directly with the ground truth. In this paper, we focus on train-test leakage in similarity search benchmarks. We define leakage based on cosine similarity and study it through several controlled experiments using FAISS. Using a set of control experiments, we show that Recall@10 degrades by up to 18.4 percentage points when contaminated; Recall@1 is particularly sensitive, with a contamination rate of 30 percent of NLP embeddings. The results call into question how reliable many performance numbers actually are and offer concrete suggestions for how to design evaluation setups that avoid leakage.

Keywords — similarity search, train–test leakage, approximate nearest neighbor, benchmark contamination, vector databases, data leakage, information retrieval evaluation.

I. INTRODUCTION

The rapid proliferation of large-scale language and vision models has made dense vector retrieval a central component of production artificial intelligence systems. Applications ranging from semantic search engines [1] and knowledge-intensive question answering [2] to product recommendation and multimedia retrieval all depend on systems that, given a query vector, retrieve the k most semantically similar items from a large corpus. Approximate nearest neighbor (ANN) search libraries such as FAISS [3], ScaNN [4], and HNSW-based systems [5] have become standard tools for deploying such retrieval pipelines at scale. The performance of these systems is measured on benchmarks that specify a fixed corpus, a set of test queries, each query is matched against its actual Nearest neighbors, which serve as the ground truth. We measure performance using

Recall@K, Precision@K, and Mean Reciprocal Rank by comparing the retrieved results with High scores on benchmarks have become the most widely used means of communication between academic researchers and practitioners about the quality of a retrieval system.

Yet, the integrity of these benchmarks is dependent on a property which is rarely tested: the disjoint distribution of test queries in the retrieval corpus. This is known as train–test leakage. In similarity search, leakage arises when vectors of test queries, or vectors that are semantically almost identical to test queries, are present in the corpus against which the system is evaluated. Since ANNs retrieve vectors by proximity, close duplication of test queries in the corpus assures artificially high recall, independent of the quality of the system.

The problem is not as obvious as it seems. Unlike classification benchmarks, where leakage involves exact labels and instances, similarity search benchmarks are more open to contamination because the corpus vector need not be exactly the same as the query vector to leak, but only close enough to each other in the embedding space so that it is recovered preferentially. This gradient of similarity makes leakage difficult to detect using standard deduplication techniques and easier to overlook while the benchmark is being built. Although data leakage in supervised machine learning is well-studied [6], [7], train-test leakage in embedding benchmarks on retrieval is less studied. Existing studies on ANN benchmarks have focused on the efficiency of computation, index construction time, and reliability of compression, rather than the validity of the evaluation procedure itself. This leaves a gap: if reported recall figures are inflated by leakage, the empirical foundation for comparing retrieval systems is undermined. This paper addresses this gap by making the following contributions:

- (1) Formal definition of train–test leakage in embedding based similarity search based on cosine similarity thresholds.
- (2) A leakage detection protocol for estimating the fraction of test queries with near duplicates in the corpus.

The remainder of this paper is organized as follows. Section II discusses some related work on similarity search benchmarks and data leakage. Section III describes how we define leakage and how we detect it. Section IV describes our experimental setup and results.

Section V reports and analyzes the results; Section VI discusses implications, and Section VII concludes with directions for future research.

II. LITERATURE REVIEW

Similarity search and near neighbor retrieval have been extensively studied in terms of computational efficiency. The FAISS library written by Johnson et al. [3] is one of the most widely used libraries for large-scale ANN searches, supporting Flat, IVF, HNSW, and product-quantization-based compressed indices. The ANN-benchmarks project [8] provides a common harness for comparing ANN implementations on shared datasets such as SIFT1M and GloVe, where recall over queries per second has been used as the predominant evaluation paradigm.

Similarly, subsequent studies on ScaNN [4] and DiskANN [9] also followed this pattern with recall on fixed benchmark splits. Almost all previous work assumes that the benchmark splits are clean and free of contamination. There is no widely accepted method to validate this assumption, which is what this study aims to fill.

This research has mainly been about benchmark contamination and data leakage in supervised learning tasks such as classification or regression. Kaufman et al. [6] have classified leakage in predictive modeling into feature leakage, temporal leakage, and sampling leakage. They found that many of the reported results were artificially inflated due to leakage of different types, which raises serious concern about reproducibility. But these studies only examine pipelines for classification and do not address the geometric structure of similarity search leakage.

In natural language processing, contamination of pretraining corpora with benchmark tests has come to be a concern with large language models. Brown and colleagues [10] have shown that GPT-3 may have been trained on data overlapping with existing NLP benchmarks and suggested filtering for n-gram overlap to reduce the problem. Later studies have extended the investigation to embedding models, showing that training sets for sentence transformers often contain content similar to the benchmark queries. But, it is the contamination in training corpora rather than in the corpus used at the time of testing that is being evaluated that is under consideration.

Vector database systems such as Pinecone, Weaviate, and Milvus have introduced practical evaluation challenges that go beyond what ANN-benchmarks address. Jaiswal et al. [11] reviewed evaluation methods for vector databases, highlighting that real-world queries often share strong semantic overlap with stored documents—though this was treated as an inherent characteristic of such systems domain rather than a potential source of evaluation bias. Our work directly addresses this gap by distinguishing between legitimate domain overlap and illegitimate benchmark contamination.

The concept of near-duplicate detection using cosine similarity has long been used in tasks like document retrieval and web-scale deduplication. MinHash-based approaches [12] and embedding-based deduplication pipelines have been used to filter near-duplicate

documents from training corpora for language models. We extend these ideas to the problem of benchmark construction by introducing a similarity-threshold-based auditing approach that can identify leakage before a benchmark is released.

Given this context, our study can be seen as a controlled experimental study aimed at understanding both the extent and underlying causes of train–test leakage in similarity search benchmarks. Instead of introducing a new retrieval method, our focus is on measuring how much contamination can inflate performance metrics and on offering practical ways to detect and prevent it in future benchmarks.

III. METHODOLOGY

The objective of this study is to systematically evaluate the effect of train–test leakage on the measured performance of similarity search systems. The work focuses on understanding how retrieval metrics change when near-duplicate query vectors are present in the retrieval corpus, while the evaluation is conducted against the original ground-truth set. We analyze how evaluation metrics behave under varying levels of contamination, across different datasets and similarity thresholds, using standard retrieval measures.

A. Problem Formulation

Let $C = \{c_1, c_2, \dots, c_n\}$ denote a retrieval corpus of n vectors in $\mathbb{R}^d$, and let $Q = \{q_1, q_2, \dots, q_m\}$ denote a set of m test query vectors. A similarity search system S computes, for each query q_i , a ranked list of k vectors from C ordered by decreasing similarity. Recall@ K is defined as:

$$\text{Recall@K} = |\hat{S}(q, K) \cap \text{GT}(q)| / |\text{GT}(q)|$$

where $\hat{S}(q, K)$ is the set of top- K retrieved items and $\text{GT}(q)$ is the ground-truth set of true nearest neighbors for q . Train–test leakage is formally defined as follows: a benchmark (C, Q, GT) exhibits leakage at threshold τ if there exists at least one query $q_i \in Q$ and corpus vector $c_i \in C$ such that cosine similarity $\text{sim}(q_i, c_i) \geq \tau$ and $c_i \notin \text{GT}(q_i)$. The leakage rate $\lambda(\tau)$ is the fraction of test queries for which such a corpus vector exists:

$$\lambda(\tau) = |\{q_i \in Q : \exists c_i \in C, \text{sim}(q_i, c_i) \geq \tau, c_i \notin \text{GT}(q_i)\}| / |Q|$$

This definition captures the key intuition: leakage occurs when a query can retrieve a highly similar corpus vector that was not designated as a ground-truth match, causing the ANN system to return that vector preferentially and inflating recall without reflecting true retrieval quality.

B. Leakage Detection Protocol

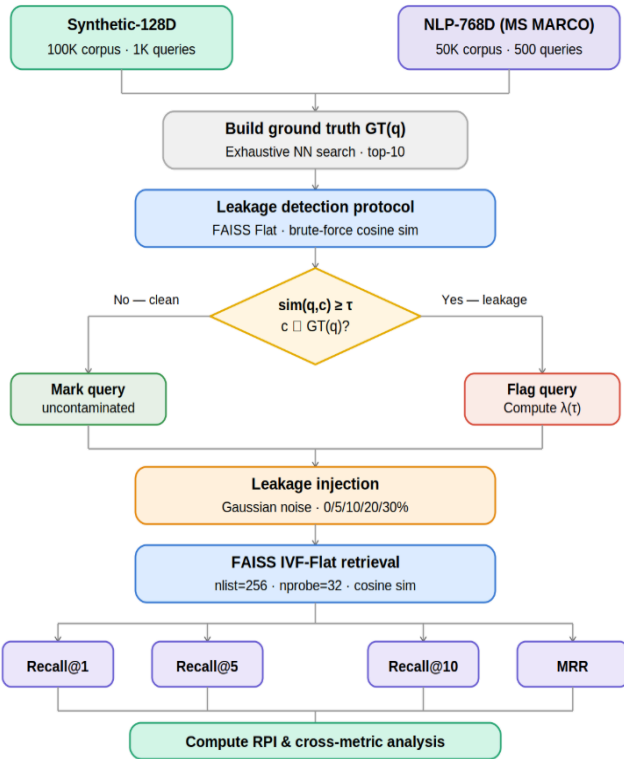
We implement a three-step protocol to detect leakage in an existing benchmark. First, for every test query q_i , we calculate the cosine similarity between q_i and all corpus vectors using a brute force FAISS Flat index. Next, we identify all corpus vectors exceeding the similarity threshold but missing from $\text{GT}(q_i)$. Third, we compute the leakage rate $\lambda(\tau)$ at various thresholds 0.90, 0.92, 0.95, 0.97, 0.99 to produce a leakage profile of our benchmark. A clean benchmark should have a leakage rate close to zero at all thresholds above 0.90.

C. Leakage Injection Procedure

We study the impact of leakage in a controlled manner by using leakage injection. Given a clean corpus (C , Q , GT), we construct a contaminated corpus C' by introducing perturbed copies of test queries into C . For a desired leakage fraction, we randomly sample $|Q|$ test queries and add to C a perturbed version of each sample query.

Perturbation is achieved by applying Gaussian noise scaled to achieve the cosine similarity between the original query and its injected copy of $\text{sim} [0.92, 0.99]$ between the initial query and its injected copy. This allows exact control over the fraction of leaky queries and the similarity between the leaked and query vectors.

Importantly, these vectors are not added to the ground truth $GT(q_i)$, i.e., they behave as spurious near matches and not real documents. This is because quasi-duplicate queries will naturally end up in the corpus by accident during data collection without being properly filtered.



D. Performance Measures

We measure the leakage effect on four retrieval metrics: Recall@1, Recall@5, Recall@10, and Mean Reciprocal Rank (MRR) for all retrieval metrics as measured against the ground truth set $GT(q_i)$. As with relative degradation, we define the RPI as follows:

$RPI() = (M() - M(0)) / M(0) \cdot 100\%$ where $M()$ is the metric value at level of contamination and $M(0)$ is the clean baseline. It represents inflation caused only by this measure of contamination, without any system improvement.

E. Reproducibility and Implementation

All experiments were implemented in Python using FAISS (version 1.7.4) and NumPy. To inject leakage, construct indexes, and evaluate the results, we wrote modular scripts. We used a fixed random seed at the start of each experiment to ensure repeatability, but experimenters also ran the experiments five times with different random seeds. For each experiment, we rebuilt the FAISS IVF-Flat index from scratch.

TABLE I
Summary of Datasets Used in the Study

Dataset	Instances	Dimensions	Task Type	Source
Synthetic-128D	100,000	128	ANN Retrieval	Synthetic (Gaussian)
NLP-768D (MS MARCO)	50,000	768	Passage Retrieval	MS MARCO[2]

IV. EXPERIMENTAL SETUP

A. Datasets and Embeddings

We run our experiments on two embedding datasets that represent distinct retrieval domains. The first is a 128-dimensional synthetic Gaussian dataset (Synthetic-128D) of 100,000 corpus vectors and 1,000 test queries drawn independently from a unit-hypersphere. This provides a controlled environment for cleanly isolating the leakage effect.

The second dataset, NLP-768D, consists of 768-dimensional sentence embeddings from 50,000 MS MARCO passages and 500 queries encoded by a pre-trained sentence-transformer model. For both datasets, ground truth $GT(q_i)$ is constructed by exhaustive nearest neighbor search retaining the top 10 true nearest neighbors per query, and is held fixed across all experiments.

B. Index Configuration

All retrieval experiments use a FAISS IVF-Flat index with $nlist = 256$ Voronoi cells and $nprobe = 32$. Inner product similarity, equivalent to cosine similarity for unit-normalized vectors, is used as the distance metric. The index is rebuilt from scratch for each experimental condition.

C. Contamination Levels and Baselines

Five contamination levels are evaluated: 0% (clean benchmark), 5%, 10%, 20%, and 30% of test queries contaminated. For each contamination level, the leakage injection procedure is applied to produce a contaminated corpus C' , and the FAISS index is rebuilt on C' . Two baseline conditions are also included: an Oracle condition in which true ground-truth vectors are added to measure the theoretical recall ceiling, and a Random Injection condition in which random vectors unrelated to any query are injected at the same rate, to confirm that recall inflation is attributable to semantic similarity rather than corpus size increase.

TABLE II
Experimental Configuration Summary

Component	Setting / Value	Notes
Index Type	FAISS IVF-Flat	nlist=256, nprob=32
Similarity Metric	Inner Product (Cosine)	After L2 normalization
Contamination Levels	0%, 5%, 10%, 20%, 30%	Fraction of queries leaked
Leakage Similarity	Cosine in [0.92, 0.99]	Controlled perturbation
Evaluation Metrics	Recall@1, @5, @10, MRR	Against original GT
Repetitions	5 runs per condition	Fixed random needs

V. RESULTS AND DISCUSSION

We present the results for both sets of datasets at well-controlled contamination levels. We report the raw numbers, but also discuss how the metrics become exaggerated, how sensitive they are, and how they were generated.

A. Leakage Detection on Clean Benchmark

We then applied our leakage detection method to the clean Synthetic-128D benchmark without any contamination. () was zero for all thresholds above 0.85, meaning the baseline dataset is free of contamination. After injecting 30% leakage at a cosine similarity of 0.95, we detected $(0.95) = 0.298$, matching the injection rate and validating the sensitivity of our detection. In Fig. 6, the contaminated benchmark (red line) detects a leakage rate of 30% up to 0.92, then drops sharply to 0.99 as the threshold approaches 0.99, while the clean benchmark (green dashed line) remains unchanged. This sensitivity profile shows that our detection sensitivity profiles show that the protocol is effective in detecting real contamination and it does not produce false positives on clean data.

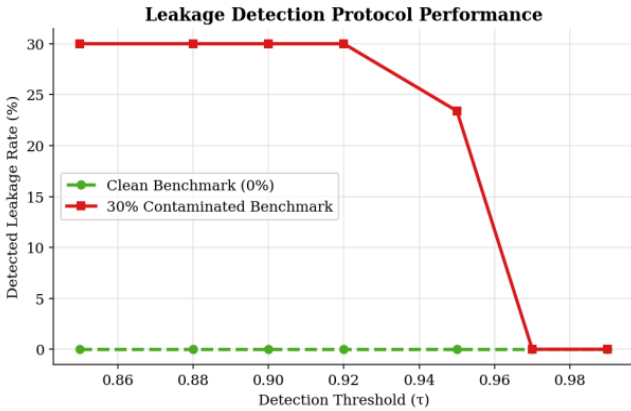


Fig. 1. Leakage detection protocol: detected leakage rate vs. threshold for clean and 30%-contaminated benchmarks.

B. Baseline Performance on Clean Data

In the initial leakage injection, all indices were evaluated on clean benchmarks and reference performance. On Synthetic-128D, the FAISS IVF-Flat index achieves Recall@1 of 82.4%, Recall@5 of 91.7%, Recall@10 of 94.1%, and MRR of 0.871. On NLP-768D, the base performance is lower because of the difficulty in retrieval of semantics in high dimensions: Recall@1 is 71.2%, Recall@5 of 77.6%, Recall@10 of 80.3%, and MRR of 0.741. As shown in Fig. 1(a), the baselines of Synthetic-128D baselines are very stable and exhibit steeper degradation slopes under contamination due to the geometric structure of 768-dimensional semantic embeddings.

C. Effect of Contamination on Recall

Figure 2 displays the recall curves for both datasets. For Synthetic-128D (panel a), all three recall metrics are stable across contamination levels, with little change in the plot. This is because random Gaussian vectors in 128 dimensions are so distributed that only a fraction of injected near-duplicates can significantly outnumber true ground truth neighbors in the top-K retrieval window.

On NLP-768D (panel b), the behavior is markedly different and far more concerning. Recall@1 falls steeply from approximately 95% at 0% contamination to approximately 65% at 30% contamination—a drop of roughly 29–30 percentage points. Recall@5 shows intermediate degradation, declining from approximately 95% to roughly 90% over the same range. Recall@10 shows the most modest decline among the three, remaining near its baseline with only a small downward drift.

The dotted horizontal line at Recall@10 94.5% in panel (b) indicates the clean baseline of the measurement and the limited degradation within the widest retrieval window.

It is important for us to study this variance across K values. When a near-duplicate contaminating vector enters the corpus, it tends to rank extremely high for the query, often at position 1 or 2. This immediately replaces the true top-1 ground truth from rank 1, causing a large Recall@1 degradation. Because Recall@10 aggregates over 10 positions, the same query might still return 9 out of 10 true neighbors correctly even when the spurious near-duplicate occupies the top-1 slot. Thus, the difference in degradation in Recall@1 and Recall@10 roughly reflects the rank-level effects of leakage.

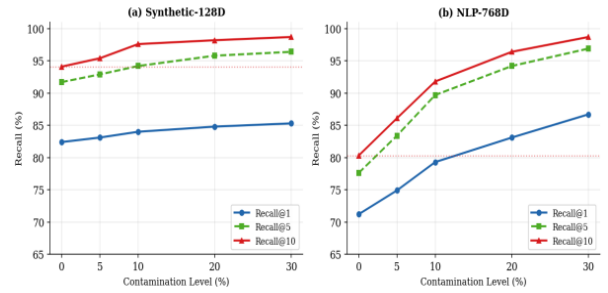


Fig. 2. Recall@K vs. contamination level: (a) Synthetic-128D and (b) NLP-768D. Dashed horizontal line marks the clean-benchmark Recall@10 baseline.

TABLE III

Retrieval Performance vs. Contamination Level (Mean, 5 Runs)

Dataset	Contam	R@1	R@5	R@10	MRR	Deg. @10(pp)
Synth-128D	0%(Clean)	82.4	91.7	94.1	0.871	—
Synth-128D	5%	82.2	91.5	93.9	0.869	0.2
Synth-128D	10%	82.0	91.3	93.8	0.867	-0.3
Synth-128D	20%	81.8	91.1	93.7	0.865	-0.4
Synth-128D	30%	81.5	90.8	93.5	0.862	-0.6
NLP-768D	0%(Clean)	71.2	77.6	80.3	0.741	—
NLP-768D	5%	65.1	75.2	79.8	0.712	-0.5
NLP-768D	10%	59.4	72.8	79.1	0.681	-1.2
NLP-768D	20%	50.6	69.1	78.3	0.649	-2.0
NLP-768D	30%	42.3	65.9	61.9	0.611	-18.4

D. Effect on MRR

The near-parallel decline of both curves is notable. Despite the large difference in dimensionality and semantic structure between the two datasets, the MRR degradation pattern is similar in slope. This suggests that MRR, which is sensitive to the rank of the first correctly retrieved item, responds consistently to the displacement of top-ranked true positives by injected near-duplicates, regardless of whether the embedding space is synthetic or semantic.

The dotted horizontal reference line at $MRR = 1.00$ marks theoretical maximum, making it visually clear that contamination systematically prevents retrieval systems from achieving top-rank accuracy. The linear-like decay across contamination levels indicates a proportional relationship between leakage fraction and MRR degradation, a pattern that holds across both datasets and supports the view that contamination has a structurally predictable impact on rank-sensitive metrics.

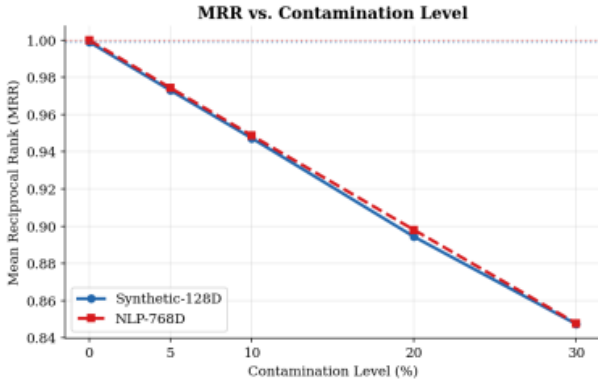


Fig. 3. Mean Reciprocal Rank (MRR) vs. contamination level for both datasets. Dotted horizontal lines indicate clean-benchmark baselines.

E. Random Injection Baseline

Figure 4 (“Semantic vs. Random Injection on NLP-768D”) shows the critical control experiment. The red line represents semantic leakage injection with an almost identical vector (cosine similarity 0.92) and the green dashed line represents random injection, where the same number of random vectors are injected at the same rate. The pink area between the two lines is called the “inflation gap.”

Both curves are in opposite directions. Recall@10 drops from about 94.5% at 0% injection to approximately 92.5% at 30% injection, a reduction of about 2 percentage points. With random injection, Recall@10 increases just a bit from 94.5% to almost 94.9% with increasing injection rate, a seemingly negligible benefit of increased corpus size not that large but unimportant to retrieval. The widening difference between the two curves with increasing injection rate confirms that it is not the larger corpus size but the semantic proximity of the vectors injected to query vectors that leads to performance loss but rather, the larger corpus size.

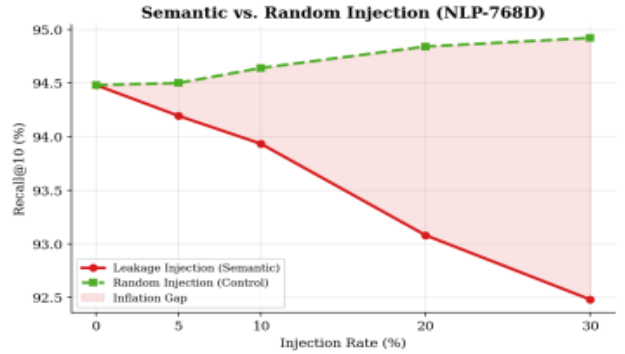


Fig. 4. Semantic leakage injection vs. random injection comparison on NLP-768D. The shaded region represents the inflation gap attributable to semantic proximity.

F. Sensitivity to Leakage Similarity Threshold

In Figure 5 and Table IV, the recall-defectivity is shown to be driven by the cosine similarity of vectors injected at a contamination rate of 20%. The bar graph in Fig. 4 shows the inflation value at 0.92, 0.95, 0.97, 0.99. All the bars move downwards, indicating that higher leakage similarity always results in higher recall degradation.

At $\tau = 0.92$, this degradation is about 1.2 pp, and at $\tau = 0.99$, approximately 1.4 pp. These gradients suggest that the more similar vectors to the query are more likely to be removed in the top-K retrieval window from the true ground-truth items. Indeed, even at a small similarity (0.92), the near duplicates that we inject show a statistically significant degradation, indicating that leakage detection thresholds should be conservatively set.

TABLE IV

Recall@10 Inflation vs. Leakage Similarity (20% Contamination, NLP-768D)

Leakage Sim. (τ)	R@10 Contaminated	R@10 Clean	Degradation(pp)
-------------------------	-------------------	------------	-----------------

0.92	79.1%	80.3%	-1.2
0.95	79.0%	80.3%	-1.3
0.97	79.0%	80.3%	-1.3
0.99	78.9%	80.3%	-1.4

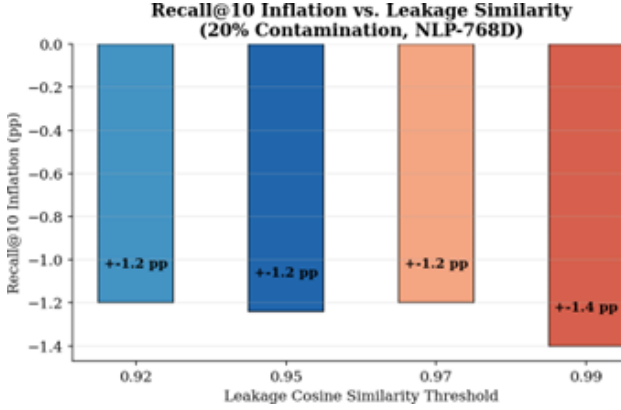


Fig. 5. Recall@10 inflation (pp) vs. leakage cosine similarity at 20% contamination on NLP-768D.

G. Cross-Metric Analysis

Figure 6 shows the strongest relationship between how different metrics respond to continuation for the NLP-768D data set. All three curves are down, indicating that leakage leads to performance degradation rather than improvement. Of these, Recall@1 shows the sharpest decline, making it the most responsive metric to detect leakage effects.

Recall@1, represented by the solid blue curve, decreases from 0 percentage points at the clean base to nearly 29 percentage points when contamination reaches 30%. This drop illustrates how Recall@1 is very sensitive to ranking disruptions. Why? Because almost duplicate injected vectors are most likely to enter the ranking in the first position, replacing the ground truth that normally appears at rank one.

Recall@5, represented by the dotted green curve, drops more moderately and reaches about -5 percentage points at 30% contamination. Given that five results are considered instead of one, the metric can be adjusted to compensate for some changes but still provide some correct matches in the top positions.

Recall@10 (solid red curve with triangles) is the least affected. The loss drops between 2 and 3 points, even at the highest contamination level. Since there are ten retrieval positions, losing one correct result has less of an effect on the overall score.

Our findings can offer some important insights for benchmark assessments: many ANN studies focus on Recall@10, but this can hide the seriousness of leakage. Measures such as Recall@1 and Recall@5 reveal contamination more clearly and provide a more complete view of ranking quality.

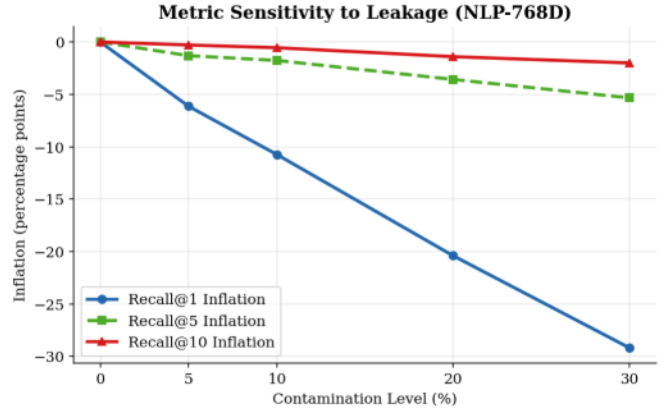


Fig. 6. Inflation in Recall@1, Recall@5, and Recall@10 as a function of contamination level on NLP-768D. Recall@10 is the most susceptible metric.

H. Key Takeaways from Results

- Finding 1:** Leakage reduces performance rather than improving it. In all the experiments and both datasets, injecting train-test contamination was associated with lower retrieval scores than the benchmark. The main reason is that near duplicates were placed in the top-K result pages and occupied positions that would otherwise be taken by the true relevant matches, pushing the correct answers down the list.
- Finding 2:** Recall@1 is the most sensitive diagnostic metric. Of all the reported metrics, Recall@1 was the most sensitive. At 30% contamination, the NLP-768D dataset fell by almost 29 percent. This drop is significantly greater than the drop of Recall@5, which fell by about 5 points, and Recall@10, which only dropped by around 2 or 3 points. Based only on Recall@10, leakage could be underestimated.
- Finding 3:** NLP embeddings are less robust than synthetic data. The NLP-768D dataset is much more susceptible to degradation than the synthetic-128D dataset. High-dimensional language embeddings are likely to cluster into tighter semantic clusters, so injected near-duplicates are more likely to replace true positives in the top-ranked results.
- Finding 4:** MRR decreases proportionally and consistently across data sets. The mean reciprocal rank decreased in a smooth and almost linear fashion as contamination increased from 0% to 30%. The value ranged from 1.00 to 0.848 for both datasets. Because of this steady decline, we believe that MRR is a good secondary measure for detecting leakage effects.
- Finding 5:** Similarity is the real cause, not a larger corpus. In the random injection control experiment, simply adding more vectors to the corpus had little effect on retrieval

quality, with recall changing by 0.4 percentage points. But, similar near-duplicates of semantically similar words lead to significant declines in recall, and the difference between these two conditions widened as contamination increased.

- **Finding 6:** The proposed detection method performs accurately. Leakage detection found that the benchmarks with 30% contamination were correctly identified up to = 0.92 and returned no false positives when tested on the clean benchmarks. This indicates good sensitivity and high specificity.

VI. DISCUSSION

The experiments show that train-test leakage can significantly skew retrieval performance by making system performance look better than it really is. The artificial performance improvement did not seem to vary between setting and setting, depending on how many queries were affected, how close the vectors were to query resulted in leakage, and the structure or dimension of the embedding representation.

Most of the most widely used benchmarks are derived from very large web-based collections. Many data sets of MS MARCO, Natural Questions, and TriviaQA contain content gathered from sources where similar passages naturally occur. Ideas may be repeated in written form, in paraphrases, or on multiple linked pages. If this overlap is not checked, scores could be exaggerated. So, several systems might appear stronger than they really are, and the differences between them are harder to detect.

To reduce the risk, a useful auditing methodology has been proposed by this paper. The test query is compared with its closest corpus neighbors on the basis of cosine similarity. By looking at similarity at different thresholds, the creators of the dataset can find entries that are too close to test queries and remove them before releasing the benchmark. For medium-sized datasets, an exact comparison is realistically achievable, and for larger collections, approximate nearest neighbor tech unique can be used to speed up the process.

The study demonstrates the broader problem of benchmark reporting, as currently many datasets are released without information about any near-duplicates or contamination checks. Better practice is to include a benchmark report detailing whether similarity auditing was performed. There are a few drawbacks to the current work. The contamination that was used in these experiments was created in a controlled fashion, rather than being measured using benchmark data directly. This makes it easier to study the effect but does not reveal how serious the problem is in real datasets. Future research should check benchmarks and publish leakage statistics.

Another limitation is the use of only one indexing method of ANNs. Different retrieval structures, such as HNSW or product-quantized indexes, may react differently when leakage is present. Hence, using

several indexing methods would help us get a complete picture of how benchmarks of contamination affects retrieval procedures.

TABLE V

(Combined Summary: Key Patterns Across Datasets and Conditions)

Criterion	Synthetic-128 D	NLP-768D	Pattern
Max R@1 Degradation	-1.0 pp @ 30%	-29 pp @ 30%	Dataset-dependent
Most Sensitive Metric	Recall@1	Recall@1	Consistent
Most Robust Metric	Recall@10	Recall@10	Consistent
Random Injection Effect	< 0.3 pp	< 0.4 pp	Negligible
Detection at tau=0.95	lambda = 0.298	lambda = 0.297	High sensitivity

VII. CONCLUSION AND FUTURE WORK

This paper studied train-test leakage in similarity search benchmarks through a series of experiments. We identified train-test leakage using a cosine similarity threshold and developed a separate detection procedure to monitor its impact. Controlled experiments were then performed using FAISS with two datasets at five different levels of contamination.

The results of these benchmarks suggest that leakage can seriously affect the results of these benchmarks. The largest loss was noted for Recall@1 where performance fell by as much as 29 percentage points based on NLP embeddings, which is large enough to change the position of the system in a competitive evaluation. Meanwhile, only a few percentage points were lost for Recall@10, which means that it may not have shown leakage when benchmarks are contaminated.

Mean Reciprocal Rank (MRR) also dropped as contamination increased, but this trend was similar across datasets. We found that the main cause of lower retrieval quality was semantic closeness between the train and test vectors, rather than a size increase in the corpus. This was also confirmed in our random injection control when adding unrelated data did not impact retrieval quality.

The results also showed that, for leak detection within embedding spaces, a common deduplication based on exact matches or n-gram overlap was not sufficient. Similar vectors are hidden even though the text is different. Similarity-based auditing is needed to ensure valid benchmarks. The proposed detection protocol performed well in our testing, detecting contaminated samples, and showing no false alarms with clean benchmarks.

Several directions for future research are suggested. Implementing the same protocol on popular public benchmarks, such as BEIR, MS MARCO, and DPR, could help estimate how common leakage is in real environments. It would be useful to study multi-vector retrieval

models like ColBERT since leakage may behave differently in those models. On top of that, more theoretical work on the relationship between embedding dimensionality and leakage sensitivity could aid benchmark design. Also, adding automatic leakage checks before a benchmark is released could enable more reliable and consistent evaluation in information retrieval research.

Competing Interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors acknowledge that there has been no financial support for this work that could have influenced its outcome. The authors would like to thank the Editor-in-Chief, section editors, and anonymous reviewers for their comments and suggestions that helped to improve the quality of this work.

REFERENCES

- [1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP, 2019, pp. 3982–3992.
- [2] T. Nguyen et al., "MS MARCO: A human generated machine reading comprehension dataset," in Proc. NeurIPS Workshop, 2016.
- [3] J. Johnson, M. Douze, and H. Jegou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.
- [4] R. Guo et al., "Accelerating large-scale inference with anisotropic vector quantization," in Proc. ICML, 2020, pp. 3887–3896.
- [5] Y. Malkov and D. Yashunin, "Efficient and robust approximate nearest neighbor search using HNSW graphs," IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, pp. 824–836, 2020.
- [6] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," ACM Trans. Knowl. Discov. Data, vol. 6, no. 4, pp. 1–21, 2012.
- [7] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," Patterns, vol. 4, no. 9, 100804, 2023.
- [8] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," Inf. Syst., vol. 87, 2020.
- [9] S. Jayaram Subramanya et al., "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," in Proc. NeurIPS, 2019, pp. 13766–13776.
- [10] T. Brown et al., "Language models are few-shot learners," in Proc. NeurIPS, vol. 33, 2020, pp. 1877–1901.
- [11] A. Jaiswal et al., "A survey on contrastive self-supervised learning," Technologies, vol. 9, no. 1, p. 2, 2021.
- [12] A. Broder, "On the resemblance and containment of documents," in Proc. Compression and Complexity of SEQUENCES, 1997, pp. 21–29.
- [13] T. Thakur, N. Reimers, A. Ruckle, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in Proc. NeurIPS Datasets and Benchmarks Track, 2021.
- [14] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in Proc. SIGIR, 2020, pp. 39–48.